

BACKEND Core Architecture & API Endpoint Map

RFT Platform NestJS Backend API

A detailed systems architecture documentation mapping database entities, implemented authentication services, university records modules, and listing the precise REST API endpoint maps required to integrate both the Administrator Portal and Student Mobile Client repositories.

BACKEND ARCHITECTURE SCOPE:

- * Database Schema - Prisma ORM, PostgreSQL Models & Enums
- * Live Auth Modules - JWT Bearer Session Tokens, Revocation & OTP Validators
- * Live Universities API - CRUD mappings with Super Admin clearance gates
- * Mobile Client Endpoint Roadmap - Courses, Library & Representative Feeds
- * Administrator Endpoint Roadmap - Super/School metrics & Intake queues

DOCUMENT METADATA

Author: Core Backend Team
Version: 1.0.0 (API Guide)
Date: June 2026

NESTJS BACKEND SYSTEM STACK

Backend Framework: NestJS 11.0.1 (Express-based)
Database Layer: Prisma Client 7.5.0 / PostgreSQL
Access Security: Passport JWT / Bcrypt Encryption

1. Executive Summary & Architecture

Executive Summary

The RFT Backend System acts as the single source of truth for the RFT edutech ecosystem, serving both the Next.js Administrator Portal and the Expo Mobile Client. Built on the NestJS framework, it implements structured REST API endpoints, secure JSON Web Token (JWT) credentials mapping, revocation registers, and automated one-time password (OTP) queues for email validation and recovery.

Data access is managed through the Prisma Object-Relational Mapper (ORM), communicating with a PostgreSQL relational database. The database entities map institutional roles (Students, Class Representatives, Lecturers, and Administrators) under a single inherited User structure, allowing immediate role transformations and unified access validation checks.

Directory Structure Overview

The repository uses NestJS modular filesystems to group resource modules, controllers, DTO schemas, and services:

Path / File	Technical Purpose & Responsibility
/src/main.ts	Application bootstrapper registering validation pipes, global filters, and CORS configs.
/src/app.module.ts	Root module loading configuration files, Prisma service nodes, and active modules.
/src/prisma	Service wrapper for the active database connection (PrismaService.ts).
/src/auth	Authentication controllers, password guards, JWT strategies, and sign-up/login DTOs.
/src/universities	Management controller mapping university CRUD fields and administration endpoints.
/prisma/schema.prisma	Prisma DB schemas defining Enums (Role, OtpPurpose) and Models.
/prisma/seed.ts	Initial seeder script generating 15 universities and a default SUPER_ADMIN user.
/postman.json	Standard Postman Collection mapping implemented routes for developers to test local requests.

2. Database Schema & Live Endpoints

The database models are designed to represent university frameworks and manage security sessions. The schema contains the following main models:

- **University**: Stores campus records. Fields: id (Int PK), name (String Unique), country (String), courseRepAccessCodeHash (String for rep elevation), createdAt, and updatedAt.
- **User (Mapped to "Student")**: Unified table housing all system profiles (Student, Course Rep, Lecturer, Admin, Super Admin). Fields: id, name details, email (Unique), phone, passwordHash, role, matricNumber (Unique), universityId (FK), and timestamps.
- **Otp**: Handles one-time verification tokens. Fields: id, purpose (VERIFY_EMAIL, RESET_PASSWORD), codeHash, expiresAt, consumedAt, and userId (FK referencing User).
- **RevokedToken**: Revocation registry for invalidating JWTs (e.g. during user logouts) by checking token identifiers (jti).

Live Implemented Endpoints

The following routes are implemented, integrated with the PostgreSQL database, and ready for connection:

Verb	Endpoint Path	Action / Role	Access
POST	/auth/register	Register standard Student or Class Representative account.	Public
POST	/auth/login	Authenticate credentials and return JWT bearer access token.	Public
POST	/auth/verify-email	Verify account email using Otp code check.	Public
POST	/auth/forgot-password/request	Request OTP token reset link via recovery email.	Public
POST	/auth/forgot-password/reset	Reset password using valid OTP check.	Public
GET	/auth/me	Fetch active profile data (role, details). Guarded by JWT/Revocation	Guarded
POST	/auth/register/lecturer	Provision new lecturer credentials. Restricted to ADMIN.	Admin Guard
POST	/auth/super-admin/create-univ-admin	Create university admin profiles. Restricted to SUPER_ADMIN.	Super Guard
POST	/auth/logout	Revoke JWT bearer token using JTI cache registries.	Guarded
GET	/universities	List all onboarded universities. Supports country query filter.	Public
POST	/universities	Create a new university record. Restricted to SUPER_ADMIN.	Super Guard
PATCH	/universities/:id	Update university metadata fields. Restricted to SUPER_ADMIN.	Super Guard
DELETE	/universities/:id	Delete university node. Restricted to SUPER_ADMIN.	Super Guard

3. API Endpoint Roadmap: Student Mobile Client

To replace the static mock data in the mobile app, the backend must implement the following endpoints. Each endpoint maps directly to student and class rep screens:

1. Courses & Study Materials (CoursesScreen.tsx)

- GET /courses : Returns all courses enrolled by the active student (derived from their universityId). Supports query parameters for text search (?q=) and department filtering.
- GET /courses/:id : Returns comprehensive syllabus structures, scheduling summaries, and assigned lecturer details.
- GET /courses/:id/materials : Lists syllabus documents, video recordings, and study packs. Returns secure file URLs (e.g. presigned S3 links) with read-only indicators.

2. Past Questions Library & Simulated Purchases (LibraryScreen.tsx)

- GET /past-questions : Retrieves past exam papers. Supports query filters: ?courseId=, ?level=, ?departmentId=.
- GET /past-questions/:id : Detailed past question layout including document preview fragments, pricing parameters, and download stats.
- POST /past-questions/:id/checkout : Triggers the mock payment process. Wire it to payment SDK tokens (Paystack/Flutterwave). Upon verification, saves a purchase transaction model, unlocking the file.

3. Announcements & Representative Feeds (RepScreen.tsx)

- GET /announcements : Returns active announcements feed targeted at the student's department/level. Includes likes count, author title, and list of comments.
- POST /announcements : Creates a new communication notification. Restricted to COURSE_REP role. Allows choosing title, body, and audience criteria (universityId, department, level).
- POST /announcements/:id/like : Likes/unlikes an announcement.
- POST /announcements/:id/comments : Appends comments to the announcement thread.

4. Account Profile Settings & Alerts Center

- PATCH /users/profile : Updates student information (phone number, edit fields).
- POST /users/change-password : Changes security password with current validation requirements.
- GET /alerts : Returns notifications for the student (cancellations, reminders), integrated with polling or sockets.

4. API Endpoint Roadmap: Admin Portal (Super & School)

The Next.js Administrator Portal requires comprehensive administration control paths. The following endpoints must be exposed to support Super Admin and School Admin screens:

1. Super Admin Administration Endpoints

- GET /super/dashboard-stats : Retrieves general platform KPIs (Total Universities, Lecturers count, Student Base, Revenue).
- GET /super/schools : Lists directories of onboarded institutions, showing location details, active plans, and statuses.
- POST /super/schools/onboard : multi-step intake handler for provisioning campus records, setup fees, and primary admins.
- GET /super/subscriptions : Retrieves available plans (Free, Premium, Enterprise) and recent transaction records.
- POST /super/subscriptions/plan : Configures billing parameters, prices, and allowed administrative accounts limits.

2. School Admin Operations Endpoints

- GET /school/dashboard-stats : Returns campus statistics (Students, verified Lecturers, Course roster counts).
- GET /school/faculties : Retrieves capacities, load ratios, and level metrics for local campus faculties.
- GET /school/levels : Local student counts segmented from 100L up to 500L.
- GET /school/courses : Campus course list. POST /school/courses plans and publishes new course items.
- GET /school/lecturers : Lists active lecturers. POST /school/lecturers upgrades a lecturer user or adds new verified listings.
- GET /school/class-reps : Lists designated course communication representatives. POST /school/class-reps/assign upgrades a student to COURSE_REP role after validating their university access code.

3. School Intake Admissions Queue

- GET /school/admissions-queue : Lists student applications awaiting registrar review.
- POST /school/admissions-queue/:id/review : Verifies documentation credentials, processes intakes, sets role verification codes, and triggers email notices.

4. Lecturer Operations Endpoints

- GET /lecturer/dashboard-stats : Active teaching loads, notifications, and review logs.
- POST /lecturer/schedule : Adds a new class session (dates, times, physical Hall, hybrid parameters).
- POST /lecturer/assessments : Constructs quizzes and assignment packages (Timed CBT, Portal Upload).

5. Security Policies & DB Optimizations

Security, scalability, and performance are primary goals of the backend client. The system architecture integrates the following core policies and design patterns:

1. Token Revocation Logic (revoked-token.guard.ts)

To prevent compromised JWT access tokens from remaining valid, the logout sequence invalidates token identifiers. When a user requests `/auth/logout`, the JTI (unique identifier encoded inside the JWT payload) is logged in the `RevokedToken` table alongside its expiration timestamp. The `TokenNotRevokedGuard` interceptor compares incoming JWT JTIs against this list on every protected request, blocking invalid connections immediately.

2. Password Encryption Standard

User credentials are encrypted using the `bcryptjs` algorithm. During signup, the server hashes passwords with 10 salt rounds (customizable via `BCRYPT_SALT_ROUNDS` inside `variables.env`). Plaintext credentials are never logged or saved in DB columns.

3. Database Query Optimizations & Indexing

Prisma schemas are indexed to handle intensive query loads as the platform grows. For example, the `Otp` table implements an index on `[userId, purpose]`:

```
@@index([userId, purpose])
```

This speeds up recovery and validation checks, reducing database lookup costs during signups.

4. OTP Lifetime & Validity Management

One-time passwords carry a strict 15-minute validity lifecycle (`expiresAt`). When an OTP verification is request, the server checks if `consumedAt` is null and `expiresAt` is greater than the current timestamp. Upon successful verification, `consumedAt` is set to the current date/time to prevent reuse attacks.

6. Local Verification & Running Instructions

Follow these steps to configure your local database connection and execute the NestJS development environment for verification:

1. Environment Configuration

Create a .env file in the RFT-Students-backend root directory and configure the database URL:

```
DATABASE_URL="postgresql://postgres:postgres@localhost:5432/rft_db?schema=public"
JWT_SECRET="rft_jwt_secret_key_2026"
BCRYPT_SALT_ROUNDS=10
```

2. Dependency Setup & Database Migration

Run these commands in your shell to fetch libraries, construct tables, and seed records:

```
# Install NestJS package dependencies
npm install

# Generate Prisma client and apply database migrations
npx prisma migrate dev

# Seed the database (Generates 15 Nigerian Universities & Super Admin account)
npx prisma db seed
```

3. Run Development Server

Spin up the server on watch mode:

```
npm run start:dev
```

The server initializes at <http://localhost:3000>. You can test authorization requests immediately by loading the postman.json collection in your Postman desktop client.

4. Seed Account Credentials

Default seeded super administrator credentials:

- Email: 'superadmin@example.com'
- Password: 'SuperAdmin123!'

Use this login combination to request your first JWT and test administrator endpoints.